

Objetivo

Explorar los namespaces predeterminados de Kubernetes e identificar su propósito dentro del clúster. Crear namespaces para representar ambientes de development, staging y production mediante métodos imperativos y declarativos. Desplegar recursos con el mismo nombre en diferentes namespaces y comprobar su aislamiento lógico. Configurar una ResourceQuota y un LimitRange para controlar el consumo de recursos en un namespace. Eliminar un namespace y verificar la eliminación en cascada de los recursos contenidos.

Requisitos previos

- Haber completado la Práctica 3 y conservar el clúster Minikube de tres nodos operativo.
- Tener Docker Desktop, Minikube y kubectl disponibles.
- Utilizar Visual Studio Code con Git Bash como terminal integrada.
- Comprender Deployments, Pods, Services y manifiestos YAML.
- Tener permisos administrativos sobre el clúster utilizado para las prácticas.

Duración estimada

50 minutos

Introducción

{“En esta práctica utilizarás namespaces para organizar recursos Kubernetes en tres ambientes simulados”=>“development, staging y production. Desplegarás una aplicación con el mismo nombre en cada ambiente para comprobar el aislamiento lógico, aplicarás ResourceQuota y LimitRange en development y finalizarás eliminando staging para observar la eliminación en cascada de sus recursos. La práctica se concentra en administración de namespaces; la separación lógica avanzada entre equipos se trabajará posteriormente en una práctica complementaria.”}

Instrucciones

Preparación del directorio de trabajo

En esta práctica crearás el directorio `lab4` y comprobarás que el clúster Minikube conserva los tres nodos utilizados durante el laboratorio anterior.

Crear el subdirectorio de la práctica

- **Paso 1.** Abre **Docker Desktop** y confirma que el motor se encuentra activo antes de interactuar con el clúster Minikube.
- **Paso 2.** Abre **Visual Studio Code**, selecciona **Git Bash** como terminal integrada y crea el directorio correspondiente al Lab 6.

```
mkdir -p /c/LABS/kubernetes/lab4
```

 Copiar

- **Paso 3.** Cambia la ubicación activa de Git Bash al directorio de trabajo para almacenar los manifiestos de namespaces y aplicaciones.

```
cd /c/LABS/kubernetes/lab4
```

 Copiar

- **Paso 4.** Confirma que la terminal se encuentra en la ruta correcta antes de crear los archivos de esta práctica.

```
pwd
```

 Copiar

Salida esperada:

```
/c/LABS/kubernetes/lab4
```

 Copiar

- **Paso 5.** Ejecuta los comandos siguientes para confirmar que Minikube está activo y que los nodos continúan disponibles.

```
minikube status  
kubectl get nodes
```

 Copiar

Resultado esperado:

Los tres nodos del clúster deben aparecer en estado **Ready**.

Tarea 1. Explorar namespaces del sistema

En esta tarea identificarás los namespaces creados automáticamente por Kubernetes y revisarás algunos recursos internos antes de crear namespaces propios.

Tarea 1.1. Listar namespaces existentes

- **Paso 6.** Ejecuta el comando siguiente para obtener la lista completa de namespaces disponibles en el clúster.

```
kubectl get namespaces
```

 Copiar

Salida esperada aproximada:

NAME	STATUS	AGE
default	Active	...
kube-node-lease	Active	...
kube-public	Active	...
kube-system	Active	...

 Copiar

- **Paso 7.** Ejecuta `kubectl describe` para revisar los metadatos y estado del namespace utilizado por los componentes internos de Kubernetes.

```
kubectl describe namespace kube-system
```

 Copiar

- **Paso 8.** Lista los Pods de `kube-system` para comprobar que los componentes del clúster se encuentran aislados de los workloads de usuario.

```
kubectl get pods -n kube-system
```

 Copiar

- **Paso 9.** Consulta los objetos Lease del namespace `kube-node-lease` para observar los recursos utilizados para mantener información de heartbeat de los nodos.

```
kubectl get leases -n kube-node-lease
```

 Copiar

IMPORTANTE: Los namespaces `kube-system`, `kube-public` y `kube-node-lease` forman parte de la infraestructura del clúster. No deben utilizarse para desplegar aplicaciones de usuario ni eliminarse durante las prácticas.

Tarea finalizada

Resultado esperado: Namespaces del sistema identificados correctamente, reconociendo su función y diferenciándolos de los ambientes de usuario.

Prompt de apoyo para explicar lo realizado

➤ [Explicar la Tarea 1 con IA](#)

Tarea 2. Crear namespaces para tres ambientes

En esta tarea crearás `development`, `staging` y `production`, utilizando tanto comandos imperativos como manifiestos declarativos.

Tarea 2.1. Crear development de forma imperativa

- **Paso 10.** Ejecuta el comando siguiente para crear directamente el namespace `development` mediante kubectl.

```
kubectl create namespace development
```

 Copiar

- **Paso 11.** Agrega labels descriptivos al namespace para identificar el ambiente y el equipo responsable.

```
kubectl label namespace development \
  environment=development \
  team=backend
```

 Copiar

Tarea 2.2. Crear staging y production mediante YAML

- **Paso 12.** Crea el archivo `namespaces.yaml`, donde almacenarás la definición declarativa de los otros dos ambientes.

```
touch namespaces.yaml
```

 Copiar

- **Paso 13.** Abre `namespaces.yaml` en VS Code y agrega los manifiestos siguientes para crear `staging` y `production`.

```
apiVersion: v1
kind: Namespace
metadata:
  name: staging
  labels:
    environment: staging
    team: platform
---
apiVersion: v1
kind: Namespace
metadata:
  name: production
  labels:
    environment: production
    team: platform
  annotations:
    description: "Namespace para el ambiente de
produccion"
```

 Copiar

- **Paso 14.** Valida el archivo antes de aplicarlo para comprobar que ambos recursos tienen una estructura YAML válida.

```
kubectl apply --dry-run=client -f
namespaces.yaml
```

 Copiar

- **Paso 15.** Aplica el manifiesto para crear los dos namespaces declarativos en el clúster.

```
kubectl apply -f namespaces.yaml
```

 Copiar

Tarea 2.3. Verificar los tres ambientes

- **Paso 16.** Ejecuta la consulta siguiente para confirmar que los tres namespaces están activos y disponibles.

```
kubectl get namespaces development staging  
production
```

 Copiar

Salida esperada:

NAME	STATUS	AGE
development	Active	...
staging	Active	...
production	Active	...

 Copiar

- **Paso 17.** Consulta los labels de los namespaces para comprobar que las etiquetas añadidas permiten clasificarlos por ambiente o equipo.

```
kubectl get namespaces --show-labels
```

 Copiar

Tarea finalizada

Resultado esperado: Namespaces `development`, `staging` y `production` creados y etiquetados correctamente mediante métodos imperativos y declarativos.

Prompt de apoyo para explicar lo realizado

[➤ Explicar la Tarea 2 con IA](#)



Tarea 3. Desplegar la misma aplicación en diferentes namespaces

En esta tarea desplegarás recursos llamados `webapp` y `webapp-svc` en los tres ambientes. Kubernetes permitirá utilizar los mismos nombres porque cada recurso pertenece a un namespace diferente.

Tarea 3.1. Crear los manifiestos de aplicación

- **Paso 18.** Crea tres archivos independientes para conservar la definición correspondiente a cada ambiente.

```
touch deploy-development.yaml  
touch deploy-staging.yaml  
touch deploy-production.yaml
```

[Copiar](#)

- **Paso 19.** Abre `deploy-development.yaml` y agrega una réplica de NGINX junto con un Service interno para el ambiente de desarrollo.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp
  namespace: development
  labels:
    app: webapp
    environment: development
spec:
  replicas: 1
  selector:
    matchLabels:
      app: webapp
  template:
    metadata:
      labels:
        app: webapp
        environment: development
    spec:
      containers:
        - name: nginx
          image: nginx:1.27-alpine
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: "100m"
              memory: "64Mi"
            limits:
              cpu: "200m"
              memory: "128Mi"
---
apiVersion: v1
kind: Service
metadata:
  name: webapp-svc
  namespace: development
spec:
  selector:
    app: webapp
  ports:
    - port: 80
```

 Copiar

```
targetPort: 80  
type: ClusterIP
```

- **Paso 20.** Abre `deploy-staging.yaml` y agrega la misma aplicación con dos réplicas para representar un ambiente intermedio.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp
  namespace: staging
  labels:
    app: webapp
    environment: staging
spec:
  replicas: 2
  selector:
    matchLabels:
      app: webapp
  template:
    metadata:
      labels:
        app: webapp
        environment: staging
    spec:
      containers:
        - name: nginx
          image: nginx:1.27-alpine
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: "100m"
              memory: "64Mi"
            limits:
              cpu: "300m"
              memory: "256Mi"
---
apiVersion: v1
kind: Service
metadata:
  name: webapp-svc
  namespace: staging
spec:
  selector:
    app: webapp
  ports:
    - port: 80
```

 Copiar

```
targetPort: 80  
type: ClusterIP
```

- **Paso 21.** Abre `deploy-production.yaml` y agrega tres réplicas para representar el ambiente de producción con mayor disponibilidad.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp
  namespace: production
  labels:
    app: webapp
    environment: production
spec:
  replicas: 3
  selector:
    matchLabels:
      app: webapp
  template:
    metadata:
      labels:
        app: webapp
        environment: production
    spec:
      containers:
        - name: nginx
          image: nginx:1.27-alpine
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: "200m"
              memory: "128Mi"
            limits:
              cpu: "500m"
              memory: "512Mi"
---
apiVersion: v1
kind: Service
metadata:
  name: webapp-svc
  namespace: production
spec:
  selector:
    app: webapp
  ports:
    - port: 80
```

 Copiar

```
targetPort: 80  
type: ClusterIP
```

Tarea 3.2. Aplicar los manifiestos

- **Paso 22.** Valida los tres archivos antes de crear los recursos para detectar errores de sintaxis o estructura.

```
kubectl apply --dry-run=client -f deploy-  
development.yaml  
kubectl apply --dry-run=client -f deploy-  
staging.yaml  
kubectl apply --dry-run=client -f deploy-  
production.yaml
```

 Copiar

- **Paso 23.** Aplica los tres manifiestos para crear Deployments y Services independientes en cada namespace.

```
kubectl apply -f deploy-development.yaml  
kubectl apply -f deploy-staging.yaml  
kubectl apply -f deploy-production.yaml
```

 Copiar

- **Paso 24.** Espera a que los tres Deployments queden disponibles antes de comprobar el aislamiento entre ambientes.

```
kubectl rollout status deployment/webapp -n  
development  
kubectl rollout status deployment/webapp -n  
staging  
kubectl rollout status deployment/webapp -n  
production
```

 Copiar

Tarea 3.3. Comprobar el aislamiento lógico

- **Paso 25.** Ejecuta las consultas siguientes para observar que el Deployment `webapp` existe simultáneamente en los tres namespaces sin conflicto de nombres.

```
kubectl get deployment webapp -n development
kubectl get deployment webapp -n staging
kubectl get deployment webapp -n production
```

 Copiar

- **Paso 26.** Utiliza `--all-namespaces` para obtener una vista global y comparar los mismos recursos dentro de ambientes diferentes.

```
kubectl get deployments --all-namespaces | grep
webapp
kubectl get services --all-namespaces | grep
webapp-svc
```

 Copiar

Salida esperada aproximada:

```
development  webapp  1/1  ...
production   webapp  3/3  ...
staging      webapp  2/2  ...
```

 Copiar

Concepto clave: Los namespaces proporcionan aislamiento lógico de nombres. Un Deployment llamado `webapp` puede existir en varios namespaces porque cada combinación `namespace + nombre` identifica un recurso diferente.

Tarea finalizada

Resultado esperado: Aplicación `webapp` desplegada en los tres namespaces, comprobando el aislamiento lógico y la reutilización de nombres sin conflictos.

Prompt de apoyo para explicar lo realizado

[➤ Explicar la Tarea 3 con IA](#)

Tarea 4. Controlar recursos con ResourceQuota y LimitRange

En esta tarea aplicarás controles de consumo al namespace `development`. ResourceQuota limitará los recursos totales del namespace y LimitRange definirá valores predeterminados para contenedores que no especifiquen requests y limits.

Tarea 4.1. Crear una ResourceQuota

- **Paso 27.** Crea el archivo `resource-controls.yaml`, donde almacenarás los controles de recursos del ambiente de desarrollo.

```
touch resource-controls.yaml
```

 Copiar

- **Paso 28.** Abre `resource-controls.yaml` y agrega una ResourceQuota que limite Pods, Services, CPU y memoria disponibles para el namespace.

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: development-quota
  namespace: development
spec:
  hard:
    pods: "5"
    services: "3"
    requests.cpu: "500m"
    requests.memory: "512Mi"
    limits.cpu: "1000m"
    limits.memory: "1Gi"
```

 Copiar

- **Paso 29.** Aplica el manifiesto y revisa el consumo actual del namespace después de considerar el Deployment y Service ya existentes.

```
kubectl apply -f resource-controls.yaml
kubectl describe resourcequota development-quota
-n development
```

 Copiar

Tarea 4.2. Crear un LimitRange

- **Paso 30.** Agrega al mismo archivo el recurso siguiente para establecer requests y limits predeterminados para los contenedores creados en `development`.

```
---
apiVersion: v1
kind: LimitRange
metadata:
  name: development-limits
  namespace: development
spec:
  limits:
    - type: Container
      default:
        cpu: "200m"
        memory: "128Mi"
      defaultRequest:
        cpu: "100m"
        memory: "64Mi"
      max:
        cpu: "500m"
        memory: "512Mi"
      min:
        cpu: "50m"
        memory: "32Mi"
```

 Copiar

- **Paso 31.** Aplica nuevamente el archivo para crear el LimitRange junto con la ResourceQuota ya existente.

```
kubectl apply -f resource-controls.yaml
```

 Copiar

- **Paso 32.** Ejecuta el comando siguiente para revisar los valores mínimos, máximos y predeterminados configurados para los contenedores.

```
kubectl describe limitrange development-limits -
n development
```

 Copiar

Tarea 4.3. Validar la asignación automática de recursos

- **Paso 33.** Crea un Pod de prueba sin especificar `resources` para comprobar que el LimitRange inyecta automáticamente los valores definidos.

```
kubectl run test-limitrange \
  --image=nginx:1.27-alpine \
  --restart=Never \
  -n development
```

[Copiar](#)

- **Paso 34.** Consulta el Pod en YAML y revisa la sección `resources` añadida automáticamente durante su admisión.

```
kubectl get pod test-limitrange \
  -n development \
  -o yaml
```

[Copiar](#)

Resultado esperado dentro de `resources` :

```
limits:
  cpu: 200m
  memory: 128Mi
requests:
  cpu: 100m
  memory: 64Mi
```

[Copiar](#)

- **Paso 35.** Ejecuta las consultas siguientes para confirmar que ResourceQuota y LimitRange están activos en el namespace.

```
kubectl get resourcequota -n development
kubectl get limitrange -n development
```

[Copiar](#)

NOTA: ResourceQuota controla el consumo total del namespace, mientras LimitRange aplica reglas y valores predeterminados a objetos individuales. Son mecanismos complementarios.

Tarea finalizada

Resultado esperado: ResourceQuota y LimitRange configurados en `development`, limitando recursos y asignando valores predeterminados a los contenedores.

Prompt de apoyo para explicar lo realizado

➔ [Explicar la Tarea 4 con IA](#)

Tarea 5. Eliminar un namespace y validar la cascada

En esta tarea eliminarás `staging` y comprobarás que Kubernetes elimina automáticamente los recursos contenidos dentro del namespace sin afectar a los otros ambientes.

Tarea 5.1. Registrar los recursos antes de eliminar

- **Paso 36.** Ejecuta el comando siguiente para listar todos los recursos principales de `staging` antes de eliminar el namespace.

```
kubectl get all -n staging
```

 Copiar

- **Paso 37.** Comprueba que `development` y `production` continúan operativos para utilizarlos posteriormente como referencia de comparación.

```
kubectl get deployment webapp -n development  
kubectl get deployment webapp -n production
```

 Copiar

Tarea 5.2. Eliminar staging

- **Paso 38.** Ejecuta el comando siguiente para eliminar el namespace y permitir que Kubernetes procese la eliminación en cascada de los recursos namespaced contenidos.

```
kubectl delete namespace staging
```

 Copiar

- **Paso 39.** Comprueba que `staging` dejó de aparecer en la lista de namespaces activos.

```
kubectl get namespaces
```

 Copiar

- **Paso 40.** Intenta consultar nuevamente el Deployment que pertenecía a `staging` para confirmar que el namespace y sus recursos ya no existen.

```
kubectl get deployment webapp -n staging
```

 Copiar

Salida esperada:

```
Error from server (NotFound): namespaces "staging"  
not found
```

 Copiar

- **Paso 41.** Confirma que los recursos con el mismo nombre en `development` y `production` permanecen disponibles y no fueron afectados por la eliminación.

```
kubectl get deployment webapp -n development  
kubectl get deployment webapp -n production
```

 Copiar

Tarea 5.3. Ejecutar la verificación final

- **Paso 42.** Copia y ejecuta el bloque siguiente para validar automáticamente los principales resultados de la práctica.

```
echo "=== Verificacion final de la Practica 4
=== "

for ns in development production; do
    STATUS=$(kubectl get namespace "$ns" \
        -o jsonpath='{.status.phase}' 2>/dev/null)

    if [ "$STATUS" = "Active" ]; then
        echo "✅ Namespace $ns disponible"
    else
        echo "❌ Namespace $ns no disponible"
    fi
done

if kubectl get namespace staging >/dev/null
2>&1; then
    echo "❌ staging todavia existe"
else
    echo "✅ staging fue eliminado"
fi

if kubectl get resourcequota development-quota \
    -n development >/dev/null 2>&1; then
    echo "✅ ResourceQuota disponible"
else
    echo "❌ ResourceQuota no encontrada"
fi

if kubectl get limitrange development-limits \
    -n development >/dev/null 2>&1; then
    echo "✅ LimitRange disponible"
else
    echo "❌ LimitRange no encontrado"
fi

echo ""
echo "Deployments restantes:"
kubectl get deployments --all-namespaces | grep
webapp

echo "=== Fin de verificacion ==="
```

 Copiar

Salida esperada aproximada:

```
=== Verificacion final de la Practica 4 ===
```

[Copiar](#)

- ✓ Namespace development disponible
- ✓ Namespace production disponible
- ✓ staging fue eliminado
- ✓ ResourceQuota disponible
- ✓ LimitRange disponible

```
...
```

```
=== Fin de verificacion ===
```

IMPORTANTE: Conserva `development`, `production` y el clúster Minikube. La práctica complementaria siguiente profundizará en separación lógica por ambiente utilizando namespaces.

Tarea finalizada

Resultado esperado: Namespace `staging` eliminado correctamente junto con sus recursos, conservando intactos los ambientes `development` y `production`.

Prompt de apoyo para explicar lo realizado

[➤ Explicar la Tarea 5 con IA](#)

🔧 Resolución de problemas

Problema 1. Un namespace ya existe

Síntoma: `kubectl create namespace development` devuelve un error indicando que el recurso ya existe.

Causa probable: El namespace fue creado previamente durante otra ejecución del laboratorio.

Solución:

Comprueba primero su estado y labels antes de decidir si debes reutilizarlo.

```
kubectl get namespace development --show-labels
```

 Copiar

Si pertenece a una ejecución anterior y necesitas reiniciar completamente el ejercicio, elimina solamente ese namespace y vuelve a crearlo.

Problema 2. El Deployment no crea Pods después de aplicar ResourceQuota

Síntoma: El Deployment existe, pero algunas réplicas no pueden crearse y los eventos muestran `exceeded quota`.

Causa probable: La suma de requests o limits de los Pods supera los valores máximos definidos en `development-quota`.

Solución:

Consulta primero el consumo actual:

```
kubectl describe resourcequota development-quota -n  
development  
kubectl get pods -n development
```

 Copiar

Reduce réplicas o ajusta los valores de recursos del workload antes de modificar la cuota.

Problema 3. El Pod de prueba no recibe resources automáticamente

Síntoma: Al consultar `test-limitrage`, la sección `resources` aparece vacía o no contiene los valores configurados.

Causa probable: El Pod fue creado antes de que existiera el LimitRange o el recurso no fue aplicado correctamente.

Solución:

Confirma que el LimitRange existe y recrea únicamente el Pod de prueba.

```
kubectl get limitrange development-limits -n
development
kubectl delete pod test-limitrange -n development --
ignore-not-found

kubectl run test-limitrange \
  --image=nginx:1.27-alpine \
  --restart=Never \
  -n development
```

 Copiar

Problema 4. staging permanece en Terminating

Síntoma: Después de eliminar `staging`, el namespace permanece durante varios minutos en estado `Terminating`.

Causa probable: Kubernetes todavía está procesando recursos o finalizers existentes dentro del namespace.

Solución:

Consulta primero el estado del namespace y los recursos que todavía puedan existir.

```
kubectl get namespace staging
kubectl get all -n staging
```

 Copiar

En un laboratorio estándar con Minikube, espera a que Kubernetes complete la terminación antes de intentar intervenciones manuales sobre finalizers.

Problema 5. kubectl consulta el namespace equivocado

Síntoma: Un comando sin `-n` no encuentra un recurso que sabes que existe en otro namespace.

Causa probable: kubectl utiliza el namespace configurado en el contexto actual o `default` cuando no existe uno explícito.

Solución:

Consulta el contexto y utiliza el namespace de forma explícita.

```
kubectl config current-context
kubectl config view --minify
kubectl get deployments -n development
kubectl get deployments -n production
```

 Copiar

Para evitar ambigüedad durante esta práctica, utiliza `-n <namespace>` en las operaciones sobre recursos namespaced.

¡Felicidades! Has completado el Laboratorio.

Resultado final

Al finalizar la práctica habrás creado y administrado namespaces para tres ambientes, desplegado recursos con nombres repetidos sin conflicto, configurado límites de consumo mediante ResourceQuota y LimitRange y comprobado que la eliminación de un namespace elimina también los recursos contenidos.

Notas y consideraciones

- Esta práctica reutiliza el clúster Minikube de tres nodos creado en el Lab 3.
- Los namespaces development y production se conservarán al finalizar para servir como referencia en actividades posteriores.
- La comunicación DNS entre namespaces y la separación por equipos se profundizarán en la práctica complementaria 4.1.
- No elimines los namespaces del sistema default, kube-system, kube-public ni kube-node-lease.

URL de referencia

- [Namespaces](#)
- [Resource Quotas](#)
- [Limit Ranges](#)
- [kubectl Cheat Sheet](#)